Page No.
Date

Python - Day - 16 (Intermediate)

Topic - OOP

(*) Why Object-Oriented programming (OOP)?

problem with procedural programming

- Code becomes :-
  - Hard to maintain
  - Difficult to Scale
  - Messy when data + functions are Scattered

Solution : OOP

OOP bundles data + functions together into one unit → Object

Real-world analogy:

- Car
  - Data → color, model, fuel
  - functions → drive(), brake(), refuel()

(*) What is a class?

Defination

A class is a blueprint for Creating objects

Class car:

pass

- Car → class
- pass → placeholder (empty class)

(*) What is an Object?

Definition

An Object is an instance of a class

my_car = Car()

- Car() → Constructor call
- my_car → object created from class Car

One class → many Objects

(*) Attributes (Variables inside Objects)

Instance Attributes

Attributes store data specific to each object

```
class Car:
    def __init__(self):
        self.seats = 5

my_car = Car()
print(my_car.seats) # 5
```

Key Concept

- Self refers to current object
- Each object has its own copy of attributes

(*) The __init__() Method (Constructor)

what is __init__?

- Runs automatically when object is created

Page No.
Date

- Used to initialize attributes

```
class Car:
    def __init__(self):
        self.seats = 5
```

(*) Think of it as:
"what should happen when the Object is born?"

(*) Constructor with parameters
Making classes Dynamic

```
Class Car:
    def __init__(self, seats):
        self.seats = seats

my_Car = Car(4)
your_Car = Car(7)
```

Each object can have different data

(*) Methods (Functions inside Classes)
Defination
Method define behavior of an object

```
class Car:
    def drive(self):
        print("The car is moving")

my_car.drive()
```

Important Rules

- First parameter is always self
- Called using dot notation

(*) Combining Attributes + Methods

```
class Car:
    def __init__(self, seats):
        self.seats = seats

    def enter_race_mode(self):
        self.seats = 2

car = Car(5)
car.enter_race_mode()
print(car.seats)  #2
```

Methods can modify attributes

(*) The Turtle Module (OOP in Action)

Importing Turtle

```
from turtle import Turtle, Screen

tim = Turtle()
```

- Turtle → class
- tim → Object

(*) Object Attributes in Turtle

Page No.
Date

tim.shape ("turtle")
tim.color ("green")

- shape () → method
- color () → method

(*) Object Methods in Turtle

tim.forward (100)
tim.left (90)
tim.backward (50)

All turtle actions are methods of Turtle class

(*) Dot Notation (Very Important)

Syntax

Object.attribute
Object.method ()

Example

tim.forward (100)
tim.color ("blue")

This is how OOP Communication happens

(*) Python package & OOP

package

A collection of modules

import turtle
or
from turtle import Turtle

- package contain classes
- you create objects from those classes

(*) Difference: function Vs Method

function	Method
print()	tim.forward()
independent	Belongs to object
No object needed	Needs object

(*) Key OOP Terminology

Term	Meaning
class	→ Blueprint
object	→ instance
Attribute	→ { variable inside object
Method	→ { functions inside class
Constructor	{ __init__()
self	→ { Refers to current object

(*) Mental Model (Best Way to Remember)

class → factory
object → product
Attribute → properties
Method → Actions

Page No.
Date

(*) Common Beginner Mistakes (b)
(b) forgetting self
(b) Using class insted of object
(b) Calling method without ()
(b) Accessing attribute before initializin

" OOP groups data (attributes) and behavior (methods) into objects created from classes, making code reusable, Organized and Scalable.

Todays Key takes

- Class = blueprint
- Object = real thing
- __init = initializes data
- Self = Current Objects
- Dot notation Connects everything

{ (*)(*) Mini Project - Coffee Machine OOP version (*)(*) }